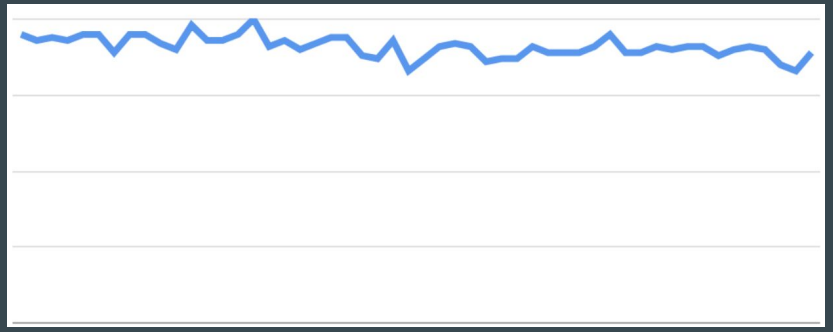
Your Flame Graph Called. AI Picked Up.

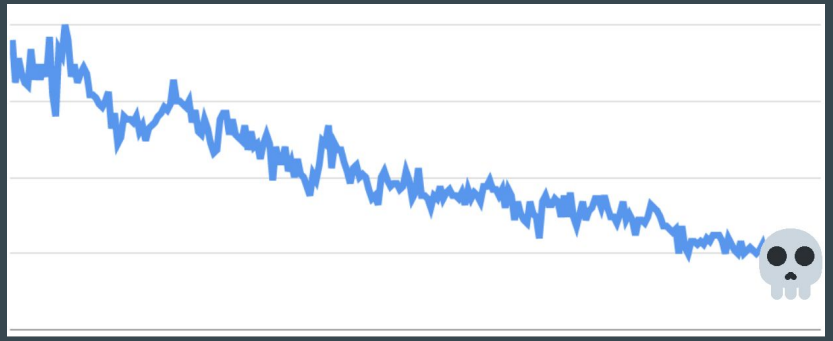
•••

Automating the Performance Bottleneck Hunt

A story

- We upgraded our traffic service JDK version from 17 to 24
- Deployed 1 instance for one day; looked okay
- Deployed the whole fleet (2 regions, ~4 instances each)
- After ~30 hours the Linux oom-killer fragged our service
- The free memory was slowly and steadily disappearing from the instances
- The investigation started...





Investigation

- Heap analysis
  - OutOfMemoryError not thrown
  - `-Xms == -Xmx`
  - `-XX:+AlwaysPreTouch`
- JFR memory profiling
  - It's great to trace heap allocations
  - It tracks native memory counts but not call sites
- Native Memory Tracking
  - the "Other" category was increasing
  - We didn't know what was allocating

```
        GC (reserved=292MB +2MB, committed=292MB +2MB)
           (malloc=20MB +2MB #29065 -15713)
           (mmap: reserved=272MB, committed=272MB)

  Compiler (reserved=2MB, committed=2MB)
           (malloc=2MB #1892 +69)

  Internal (reserved=8MB +5MB, committed=8MB +5MB)
           (malloc=8MB +5MB #42387 +4806)

     Other (reserved=1101MB +1062MB, committed=1101MB +1062MB)
           (malloc=1101MB +1062MB #139026 +136017)

    Symbol (reserved=24MB, committed=24MB)
           (malloc=22MB #351304 +2317)
           (arena=2MB #1)
```

- $ pmap PID
- $ jcmd PID VM.native_memory detail

```
0000000580000000 10476544K rw---   [ anon ]
00000007ff700000    468K rw--- classes.jsa
00000007ff775000    556K rw---   [ anon ]
00000007ff800000    476K rw--- classes.jsa
00000007ff877000   7716K rw---   [ anon ]
000000b800000000  12160K rw--- classes.jsa
000000b800be0000   4224K -----   [ anon ]
000000b801000000    256K rw---   [ anon ]
000000b801040000    768K rw---   [ anon ]
000000b801100000   1280K rw---   [ anon ]
000000b801240000   1024K rw---   [ anon ]
000000b801340000    256K rw---   [ anon ]
000000b801380000   1792K rw---   [ anon ]
000000b801540000    256K rw---   [ anon ]
000000b801580000    896K rw---   [ anon ]
000000b801660000    128K rw---   [ anon ]
000000b801680000    256K rw---   [ anon ]
000000b8016c0000    256K rw---   [ anon ]
:

0000ffffb8882000    688K r-x-- libm-2.26.so
0000ffffb892e000     76K ----- libm-2.26.so
0000ffffb8941000      4K r---- libm-2.26.so
0000ffffb8942000      4K rw--- libm-2.26.so
0000ffffb8943000     28K r-x-- librt-2.26.so
0000ffffb894a000     96K ----- librt-2.26.so
0000ffffb8962000      4K r---- librt-2.26.so
0000ffffb8963000      4K rw--- librt-2.26.so
0000ffffb8964000  17800K r-x-- libjvm.so
0000ffffb9ac6000    104K ----- libjvm.so
0000ffffb9ae0000    848K r---- libjvm.so
0000ffffb9bb4000    196K rw--- libjvm.so
0000ffffb9be5000    504K rw---   [ anon ]
0000ffffb9c63000   1456K r-x-- libc-2.26.so
0000ffffb9dcf000     64K ----- libc-2.26.so
0000ffffb9ddf000     16K r---- libc-2.26.so
0000ffffb9de3000      8K rw--- libc-2.26.so
0000ffffb9de5000     16K rw---   [ anon ]
:
```

```
Virtual memory map:

[0x0000000580000000 - 0x0000000800000000] reserved and committed 10485760KB for
    [0x0000ffffb9773d0c] ReservedHeapSpace::try_reserve_range(char*, char*, unsi
    [0x0000ffffb9774750] ReservedHeapSpace::initialize_compressed_heap(unsigned
    [0x0000ffffb9774938] ReservedHeapSpace::ReservedHeapSpace(unsigned long, uns
    [0x0000ffffb972d8fc] Universe::reserve_heap(unsigned long, unsigned long)+0x

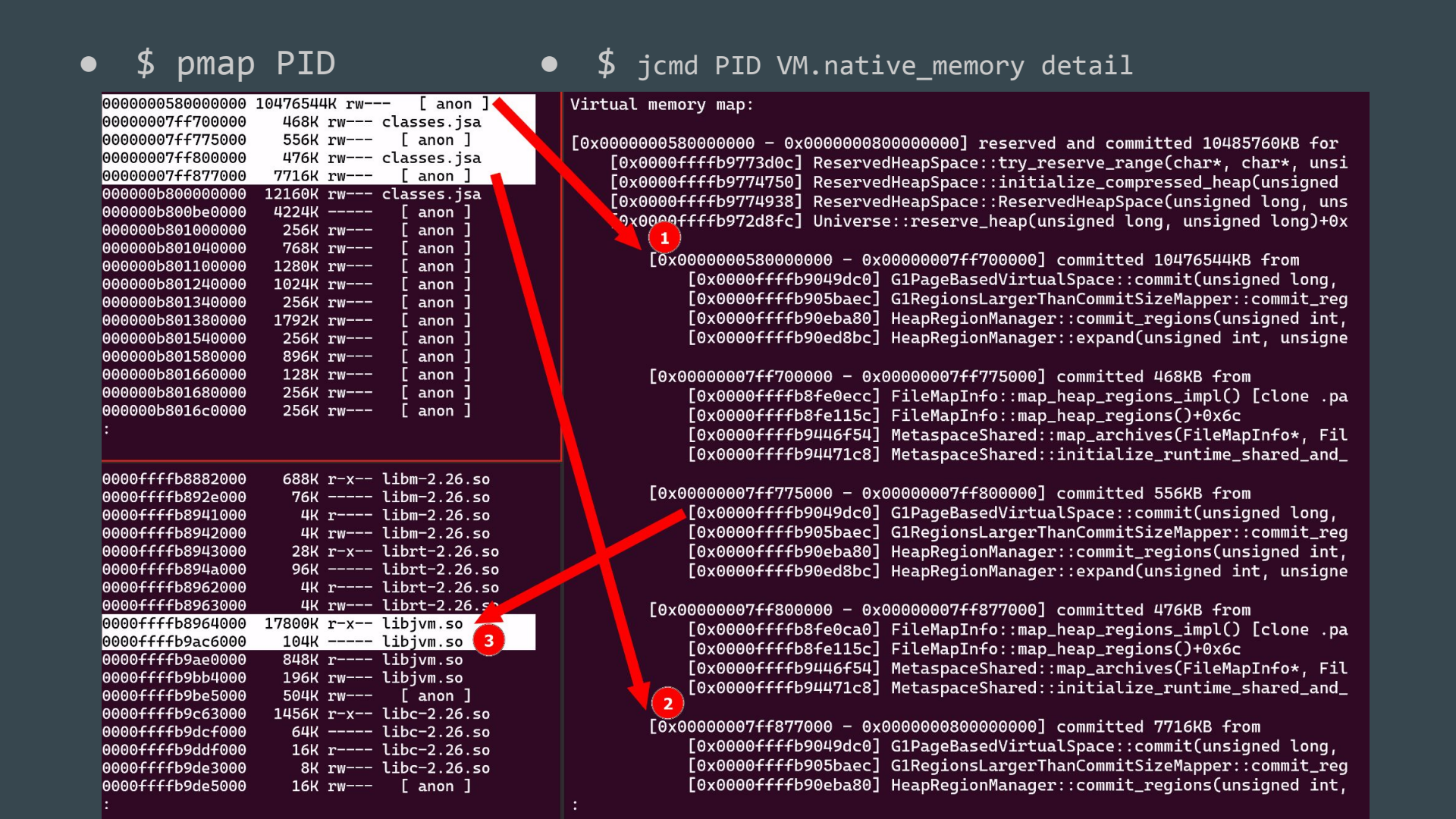
        [0x0000000580000000 - 0x00000007ff700000] committed 10476544KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
            [0x0000ffffb90ed8bc] HeapRegionManager::expand(unsigned int, unsigne

        [0x00000007ff700000 - 0x00000007ff775000] committed 468KB from
            [0x0000ffffb8fe0ecc] FileMapInfo::map_heap_regions_impl() [clone .pa
            [0x0000ffffb8fe115c] FileMapInfo::map_heap_regions()+0x6c
            [0x0000ffffb9446f54] MetaspaceShared::map_archives(FileMapInfo*, Fil
            [0x0000ffffb94471c8] MetaspaceShared::initialize_runtime_shared_and_

        [0x00000007ff775000 - 0x00000007ff800000] committed 556KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
            [0x0000ffffb90ed8bc] HeapRegionManager::expand(unsigned int, unsigne

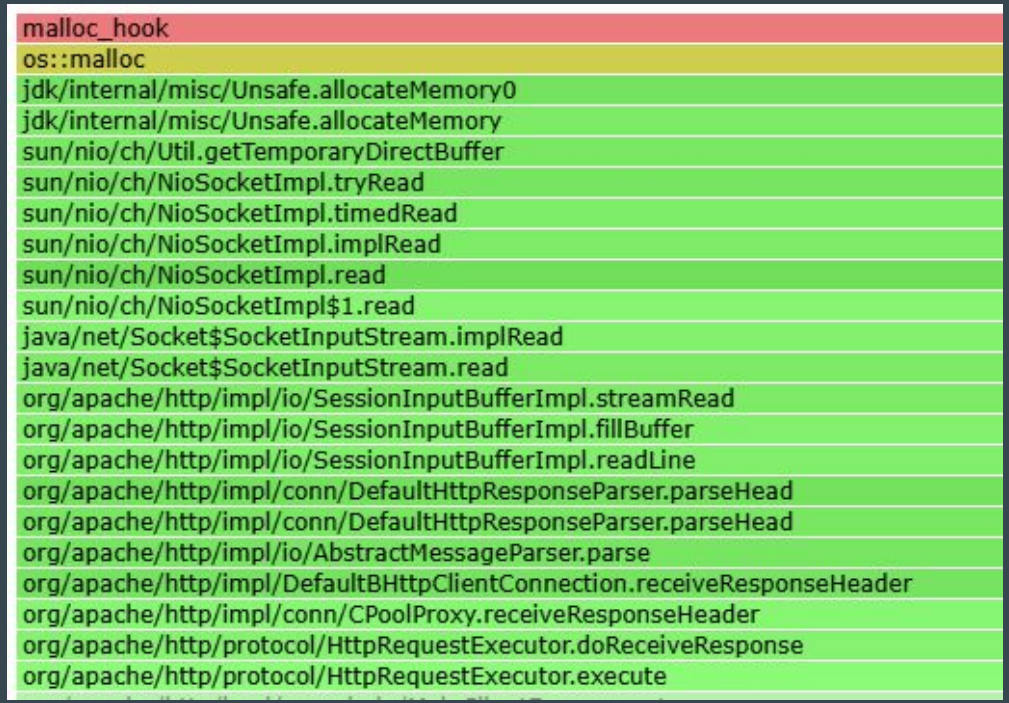
        [0x00000007ff800000 - 0x00000007ff877000] committed 476KB from
            [0x0000ffffb8fe0ca0] FileMapInfo::map_heap_regions_impl() [clone .pa
            [0x0000ffffb8fe115c] FileMapInfo::map_heap_regions()+0x6c
            [0x0000ffffb9446f54] MetaspaceShared::map_archives(FileMapInfo*, Fil
            [0x0000ffffb94471c8] MetaspaceShared::initialize_runtime_shared_and_

        [0x00000007ff877000 - 0x0000000800000000] committed 7716KB from
            [0x0000ffffb9049dc0] G1PageBasedVirtualSpace::commit(unsigned long,
            [0x0000ffffb905baec] G1RegionsLargerThanCommitSizeMapper::commit_reg
            [0x0000ffffb90eba80] HeapRegionManager::commit_regions(unsigned int,
:
```



async-profiler saved the day

- The issue cause
  - The `sun.nio.ch.Util` class
  - Native buffers cache per thread
  - Buffer count is capped, buffer size is not
  - Each thread holds a buffer with the maximum size it needed so far
  - Apache HTTP client from many threads may consume excess memory
  - `-Djdk.nio.maxCachedBufferSize=0`
- Advice
  - Be extra cautious when upgrading
  - Always monitor the metrics
  - Profile often



Summary

- Memory leaks and performance bottlenecks are hard to find
- Detect, prevent, analyze
  - Static analysis
    - Thread dumps
    - Heap dumps
    - Native Memory Tracking
  - Profiling
    - Java Flight Recorder
    - Java Mission Control
    - Async-profiler
  - Monitor and alert
    - CPU
    - Memory (heap, off-heap, OS)
    - GC
    - ...

Jacek Psiuk

- Software Engineer with 15+ years of experience, voluum.com, Commerce Media Tech, Team Internet Group
- Building distributed systems
- Computer Science Graduate, AGH University of Krakow
- Certified AWS Solutions Architect Professional



Introduction

Introduction

- Memory leaks
  - Referenced and unused memory
  - Can slowly accumulate through time
- Performance bottlenecks
  - Code that is executed the most
  - Slowest parts of program
  - May be related to memory leaks
- Common symptoms if problems
  - Decreased application performance
  - Increased resources consumption
  - Frequent or long GC collections
  - When memory runs out: OutOfMemoryErrors, OOM-killer

Common causes

- Code that is someone else's, obviously
  - Libraries we import
- Leaks
  - Unclosed resources
  - Cached objects
  - Memory management bugs in native code
  - Listeners and callbacks
  - Static references
  - Improper use of collections
- Performance bottlenecks
  - Allocating too many objects
  - Lock contention
  - Suboptimal algorithms

The goal

- We want to find performance bottlenecks efficiently
  - Customer requests processed efficiently
  - Increased system availability
  - Decreased cost
- We want to be able to use the variety of tools designed for that
- We want to have it automated with AI

Tools for static analysis used by AI

Thread dump

- Copy of each JVM thread state
- It's good to start the analysis with it
- To grasp thread's behavior in time, worth taking 2 with ~10-second gaps
- Thread dumps are long text files with many details and repetitions
- LLMs are good in text analysis and comprehension
- Three examples
  - Deadlock
  - Lock contention
  - Virtual threads

```
❯ My running java application is stuck. Get and analyze thread stack traces.
```

Heap dump

- A snapshot of objects in the heap at a certain point in time
  - May or may not contain unreachable objects
  - All objects (class, fields primitive values and references)
  - All classes (name, super class, static fields, classloader)
  - GC roots
  - Thread stacks and local variables
- We can perform the heap analysis ourselves, but this is slow...
- The snapshot is a file in a binary format - LLM is not able to digest it
- BUT, it knows a tool that is - the MAT
  - `ParseHeapDump.sh dump.hprof org.eclipse.mat.api:suspects`

```
❯ Find MAT in ~/.jdks/mat. Check the uploaded heap dump. Propose steps to fix the problem.
```

Native memory tracking - NMT

- Tracks memory off heap including used by JVM internals
  - Adds 2 machine words to all malloc memory as a malloc header
  - A 5-10% JVM performance drop
- Categories
  - Heap, Class, Thread, Code, GC, Compiler, Metaspace, NMT, Other
- How
  - `-XX:NativeMemoryTracking=detail`
  - `jcmd PID VM.native_memory detail scale=MB`
- To compare over time
  - `jcmd PID VM.native_memory baseline`
  - `jcmd PID VM.native_memory detail.diff`

```
❯ Run Ex5 with Native Memory Tracking and analyze the results. Put a summary into a dir
```

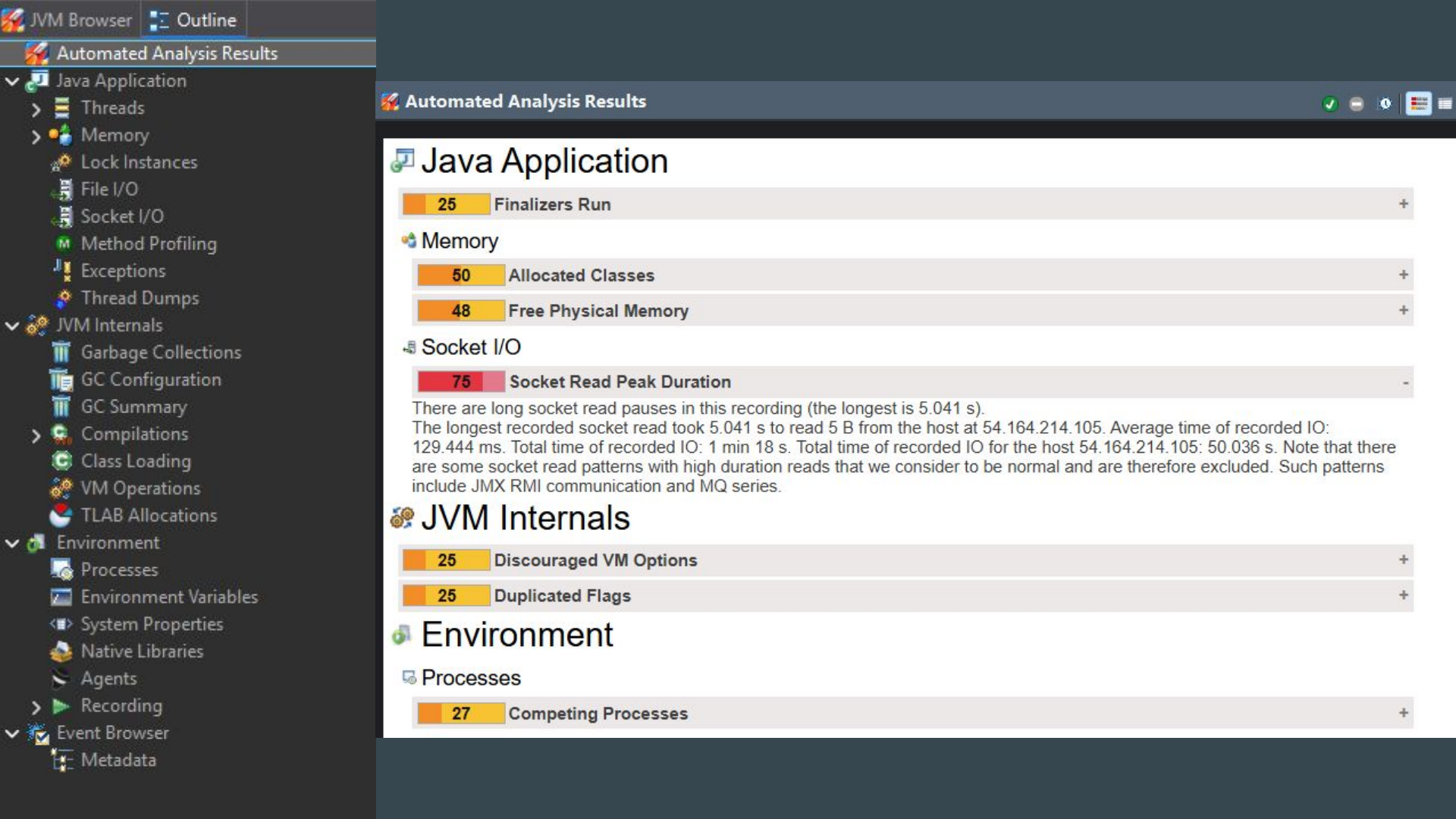
Profilers used by AI

JDK Flight Recorder - JFR

- An observability and monitoring framework built into the OpenJDK
- How
  - `jcmd <PID> JFR.start duration=10m name=profiling filename=recording.jfr settings=profile.jfc`
  - `-XX:StartFlightRecording:settings=default.jfc`
- View recording results
  - `$ jfr view [view-name or all-views] file`
  - `$ jfr view --help`
- Configuration files specify which events should be recorded
  - Two preset configurations: `default.jfc`, `profile.jfc`
  - You can create your own

JDK Mission Control - JMC

- A powerful tool for profiling and debugging applications running on a JVM
- Analyzes and visualizes JFR recordings
- Connects to JVM via JMX
  - `-Dcom.sun.management.jmxremote.port=9999`
    - Ensure no access to this port from the outside!
  - `-Dcom.sun.management.jmxremote.authenticate=false`
  - `-Dcom.sun.management.jmxremote.ssl=false`
  - `$ ssh <IP> -L 10000:127.0.0.1:9999`
  - Connect to: `service:jmx:rmi:///jndi/rmi://localhost:10000/jmxrmi`
- Flight Recorder → Start Flight Recording → Wait until it's done...
  - ...and we have a binary file with the recording


JVM Browser
Outline
Automated Analysis Results
Java Application
Threads
Memory
Lock Instances
File I/O
Socket I/O
Method Profiling
Exceptions
Thread Dumps
JVM Internals
Garbage Collections
GC Configuration
GC Summary
Compilations
Class Loading
VM Operations
TLAB Allocations
Environment
Processes
Environment Variables
System Properties
Native Libraries
Agents
Recording
Event Browser
Metadata
Automated Analysis Results
Java Application
25 Finalizers Run
Memory
50 Allocated Classes
48 Free Physical Memory
Socket I/O
75 Socket Read Peak Duration
There are long socket read pauses in this recording (the longest is 5.041 s).
The longest recorded socket read took 5.041 s to read 5 B from the host at 54.164.214.105. Average time of recorded IO: 129.444 ms. Total time of recorded IO: 1 min 18 s. Total time of recorded IO for the host 54.164.214.105: 50.036 s. Note that there are some socket read patterns with high duration reads that we consider to be normal and are therefore excluded. Such patterns include JMX RMI communication and MQ series.
JVM Internals
25 Discouraged VM Options
25 Duplicated Flags
Environment
Processes
27 Competing Processes

JDK Flight Recorder - JFR

- `$ jfr view [view-name or all-views] file`
- `$ jfr view --help`

```
Java virtual machine views:

 blocked-by-system-gc      gc-concurrent-phases jvm-information

 class-modifications       gc-configuration     longest-compilations

 compiler-configuration    gc-cpu-time          native-memory-committed

 compiler-phases           gc-parallel-phases   native-memory-reserved

 compiler-statistics       gc-pause-phases      safepoints

 deoptimizations-by-reason gc-pauses            tlabs

 deoptimizations-by-site   gc-references        vm-operations

 gc                        heap-configuration
```

JDK Flight Recorder - JFR

Java Methods that Executes the Most

Method	Samples	Percent
java.util.HashMap.getNode(Object)	437	5.80%
org.apache.tomcat.util.buf.MessageBytes.equalsIgnoreCase(String)	437	5.80%
java.lang.String.equalsIgnoreCase(String)	327	4.34%
sun.nio.cs.UTF_8$Encoder.encodeLoop(CharBuffer, ByteBuffer)	298	3.95%
java.util.concurrent.ConcurrentLinkedQueue.offer(Object)	296	3.93%
java.util.concurrent.ConcurrentLinkedQueue.size()	275	3.65%

❯ Make a profiling JFR recording for 15 seconds and analyze the result.

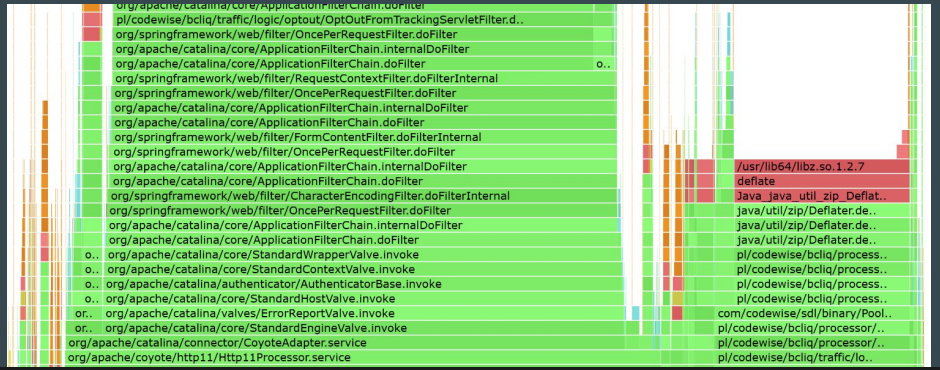
async-profiler

- A low-overhead sampling profiler for Java
- Immune to the “Safepoint bias problem”
  - Uses native stack walking to capture Java, native and kernel stack traces, ensuring a more accurate representation of the application's execution
- Monitors non-Java threads (e.g., GC and JIT compiler threads) and shows native and kernel frames in stack traces
- Maintained by the Amazon Corretto team

3 hours of YouTube presentation on async-profiler under the hood and how to use

async-profiler

- Shipped with JDK
  - `$ $JAVA_HOME/bin/asprof -d 30 -f out.html jps`
- Output
  - Textual collection of call stacks
  - Flame graphs in HTML
  - JFR recordings
- Explore
  - `$ ./asprof --help`




```
❯ Profile my running app with async-profiler. Prepare analysis results.
```

Automation

How to create an agent to automate the analysis

- Create a script to gather all the artifacts
  - Thread dumps
  - Heap dumps (optionally, depending on whether the impact is tolerable)
  - Native memory tracking (needs to be enabled, so may need an app restart)
  - JFR recording
  - async-profiler
- Give LLM access to
  - The gathered artifacts
  - Tools to analyze the data: MAT, JFR, async-profiler
  - Source code

How to create an agent to automate the analysis

- Have LLM analyze the artifacts and prepare a report with actionable items
  - Verify the content of the report
    - Summary (an action is required, recommended, none)
    - A list of detected issues sorted by severity
      - Evidence
      - Impact
      - Root cause
      - Suggested fix
      - Expected change after fix
    - Related observations
    - Analyzed artifacts and used commands
  - Verify the correctness of the report
- Repeat until the report is complete
- Compile instructions for an agent based on the performed analysis

```
❯ Write up a CLAUDE.md file with clear instructions on how to run the performance analysis
```

Defense against hallucinations

- A made-up "95% blocked in audit()" reads exactly like a computed one
- Why do models hallucinate?
- Instructions: evidence only, "unknown" is allowed, NONE is a normal answer.
- Scripts do the arithmetic, the model never does it in its head.
- Every number cites its source file.

```
❯ Tell me exactly how did you come up with the 95% number. Quote evidence.
```

Evaluate the agent, automate the evaluation

- Prepare different code examples with bugs and their fixed versions
- Write scripts to evaluate the agent
  - Assert the bugs are found correctly
  - Assert the healthy code is reported as bug-free
- Evaluate the agent before and after every change to the instructions
- Now we have proof that the agent works :-)

```
❯ /clear
❯ Run the evaluation.
❯ (adjust the instructions)
❯ Repeat until satisfied.
```

Summary

- Memory leaks and performance bottlenecks are hard to find for humans
- AI can very well automate the bottleneck hunt
- It’s easy to create a verified agent that generates a concise performance analysis report
- Create an agent, incorporate to test and production
- Ready to detect, prevent, analyze, improve



Your Flame Graph Called. AI Picked Up: Automating the Performance Bottleneck Hunt